RSCL — Algorithm 11: Mission Coordinator: Deterministic Agent Bids, Slot Negotiation, MA'AT Check, Operator Approval and Telemetry-Verified Execution

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)

Source files described (content hash and state at document build time):

hu-telepresence-gazebo/festival/mission_coordinator.py	sha256 6b85d14e1445	2026-11-11 06:42 UTC	clean (HEAD 983eede; last change 10ee643)
hu-telepresence-gazebo/festival/control_server_hu.py	sha256 3f32578a144c	2026-11-11 06:32 UTC	clean (HEAD 983eede; last change 10ee643)
hu-telepresence-gazebo/festival/formation_catalog.json	sha256 54743db49fb8	2026-11-12 01:23 UTC	modified vs b44bc1c

1. Purpose and context

The coordinator turns a high-level objective ("inspect north sector", "survey the arena", "form a wedge and hold", "relay chain to east", with constraints such as "keep Ra in reserve", "use three drones", "without Thoth", "in a V formation", "offset 1.2 m") into an ordered list of ordinary supervisor commands. The five "agents" are deterministic rule functions over telemetry (no learning, no language model). Safety authority stays with the supervisor: the plan is validated by Algorithm 10 and executed through the authorised mission gateway only after explicit operator approval.

2. Inputs, outputs, data structures

- **Objective** (≤ 200 chars) parsed by `parse_objective` to {kind ∈ inspect/survey/relay/hold, sector, shape, constraints{reserve, exclude, count, shape, offset_m}}.
- **Snapshot:** `GET /api/state` normalised per aircraft; simulated battery (100 % − 2 %/min of observed airborne time, z > 0.3 m), labelled SIMULATED_ESTIMATE.
- **Bid** per agent: available ∈ {YES, NO, UNKNOWN}, reasons, position, distance to sector point, battery, evidence references (field, value, sample time, route).
- **Plan:** steps S1..Sk with labels, MA'AT check result, assignment, transcript, four graphs (mission DAG, assignment, proximity, provenance), status ∈ {NOT_PLANNABLE, VALIDATED, REJECTED}, `executable=false`. Logged to `logs/missions/<mission_id>.jsonl`.
- **Execution report:** per step precheck, dispatch, observed outcome; status COMPLETED / STOPPED / NOT_STARTED, contingency hold.

3. Algorithm

Algorithm 11.1 PLAN

Source: festival/mission_coordinator.py: parse_objective, AgentPolicy.bid, negotiate, Coordinator.plan, _sector_offset, _alternatives

```
function PLAN(objective, end = land)
  obj ← PARSE-OBJECTIVE(objective)                        // closed grammar; else refuse
  shape ← catalog[obj.shape] must have status VALIDATED (alias resolved), else NOT_PLANNABLE
  snap ← snapshot();  P_sec ← 1.6 · unit(sector)        // north (0,1), south, east, west, center (0,0)
  for each agent n: bid_n ← BID(n)
    stale → UNKNOWN; fault or FAULT → NO; mode ∈ BUSY → NO; mode not plannable → NO;
    sim battery < 25 % → NO; reserved → NO; excluded → NO; else YES
  avail ← {n : bid_n = YES};  if count > |avail| then NOT_PLANNABLE
```

```
  team ← the count (or all) available closest to P_sec (ties: fleet order), in fleet order; require |team| ≥ 2
  C ← SLOTS(shape, |team|)                                  // Algorithm 5.1, centred
  prefs ← NEGOTIATE(team, C)                                // Algorithm 11.2 (informational)
  steps ← [takeoff(parked team members, altitude 1.7)] if any parked;  steps += [formation(team, shape)]
  chk ← PLAN-CHECK(steps)                                   // Algorithm 10; failure to reach it ⇒ REJECTED
  if chk.ok and kind ∈ {inspect, relay, survey} then offset ← SECTOR-OFFSET(obj, chk.goals of formation)
  else if explicit offset then offset ← unit(sector) · offset_m
  if kind = survey then a ← offset.y; steps += [move(0,−a), hover 6 s, move(0, 2a), hover 6 s, move(0,−a)]
  else
    if offset ≠ 0 then steps += [move(offset)]
    steps += [hover(8 s if inspect else 6 s)];  if offset ≠ 0 then steps += [move(−offset)]
  if end = land then steps += [land(team)]
  chk ← PLAN-CHECK(steps)
  assignment ← supervisor's formation goals (authoritative) mapped to centred slots; agent prefs recorded
  if chk.ok then status ← VALIDATED (executable stays false)
  else status ← REJECTED; if top level then ALTERNATIVES(...)
function SECTOR-OFFSET(obj, goals)
  if survey then return (0, 1.0);  if explicit offset then return unit·offset   // not clamped: MA'AT judges
  if sector = center or no goals then return (0,0)
  ext ← max_g (g · unit);  o ← max(0, min(1.6, 2.0, 3.10 − ext));  o ← floor(20·o)/20
  return unit · o
function ALTERNATIVES(obj, failure)                     // bounded: first 6 candidates, each fully re-planned
  candidates, ordered by the failure reason:
    drone problem (fault|telemetry|must|settled|maneuvering|is MODE;) → exclude each named participant first
    move/arena/bounds/"within 1.0 m" → drop explicit offset; offsets 0.6× and 0.3× of current if ≥ 0.2 m
    formation or move failure → shapes v, line, wedge, arc, grid, column, inverted_v (≠ current)
    otherwise exclusions last
  return first VALIDATED alternative (needs its own approval), else all tried
```

Algorithm 11.2 NEGOTIATE (bounded nearest-slot claims)

Source: festival/mission_coordinator.py: negotiate, AgentPolicy.claim_slot

```
function NEGOTIATE(team, C)
  agreed ← {};  pending ← team
  repeat at most |team| + 1 rounds while pending ≠ ∅
    claims ← for each n ∈ pending: nearest free slot k (XY distance, rounded to 0.01 m; ties by slot index)
    pending ← ∅
    for each slot k (ascending) do
      winner ← min over claimants by (distance, fleet index);  agreed[winner] ← k
      every other claimant yields (message states the distance difference or the tie rule); pending += loser
  return agreed
```

Algorithm 11.3 EXECUTE

Source: festival/mission_coordinator.py: Coordinator.execute, _verify, _done, _abort

```
procedure EXECUTE(plan, approve)
  if approve ≠ true then log EXECUTION_REFUSED; return NOT_STARTED
  if plan.status ≠ VALIDATED then return NOT_STARTED
  for i, step in steps do
    repeat pre ← PLAN-CHECK(steps[i..])                     // re-check against the live state
    until pre.step0.ok or reason ∉ {settled, maneuvering, wait for stable} or 8 s elapsed (poll 0.5 s)
    if not pre.step0.ok then ABORT(dispatched = false)
    dispatch step via /api/mission/dispatch (token, operator_confirmed, planner HU-MISSION-COORD)
```

```
    if HTTP ≠ 200 or not ok then ABORT(dispatched = false)
    obs ← VERIFY(step, pre.step0.planned_goals, timeout[action])
    if not obs.ok then ABORT(dispatched = true)
  return COMPLETED
function VERIFY(step, planned, timeout)                  // poll every 0.25 s
  loop
    track min 3D separation among fresh aircraft with z > 0.3
    if any target stale or faulted then fail
    if min separation < 0.82 then fail
    if action ∈ {formation, move} and ‖live target − planned goal‖ > 0.35 for a target then fail   // e.g. SEPARATION_HOLD
    stable ← stable + 1 if DONE(step) else 0;  if stable ≥ 3 then break
    if elapsed > timeout then fail
  if dwell_s > 0 then for dwell_s: fail if any target stale, faulted, not HOVER, or drift > 0.30 m
  return ok
DONE: takeoff — HOVER, z > 0.8, |z − planned z| < 0.2;  formation — FORMATION and within 0.10 m of setpoint;
      move — HOVER and within 0.10 m;  hover — HOVER;  land — GROUNDED/READY and z < 0.3
procedure ABORT(dispatched)
  if dispatched and action ≠ land then
    hover all fresh airborne aircraft via the mission gateway; if refused then emergency_hover (Alg. 2)
  report STOPPED with re-planning options
```

4. Flowchart and state diagram

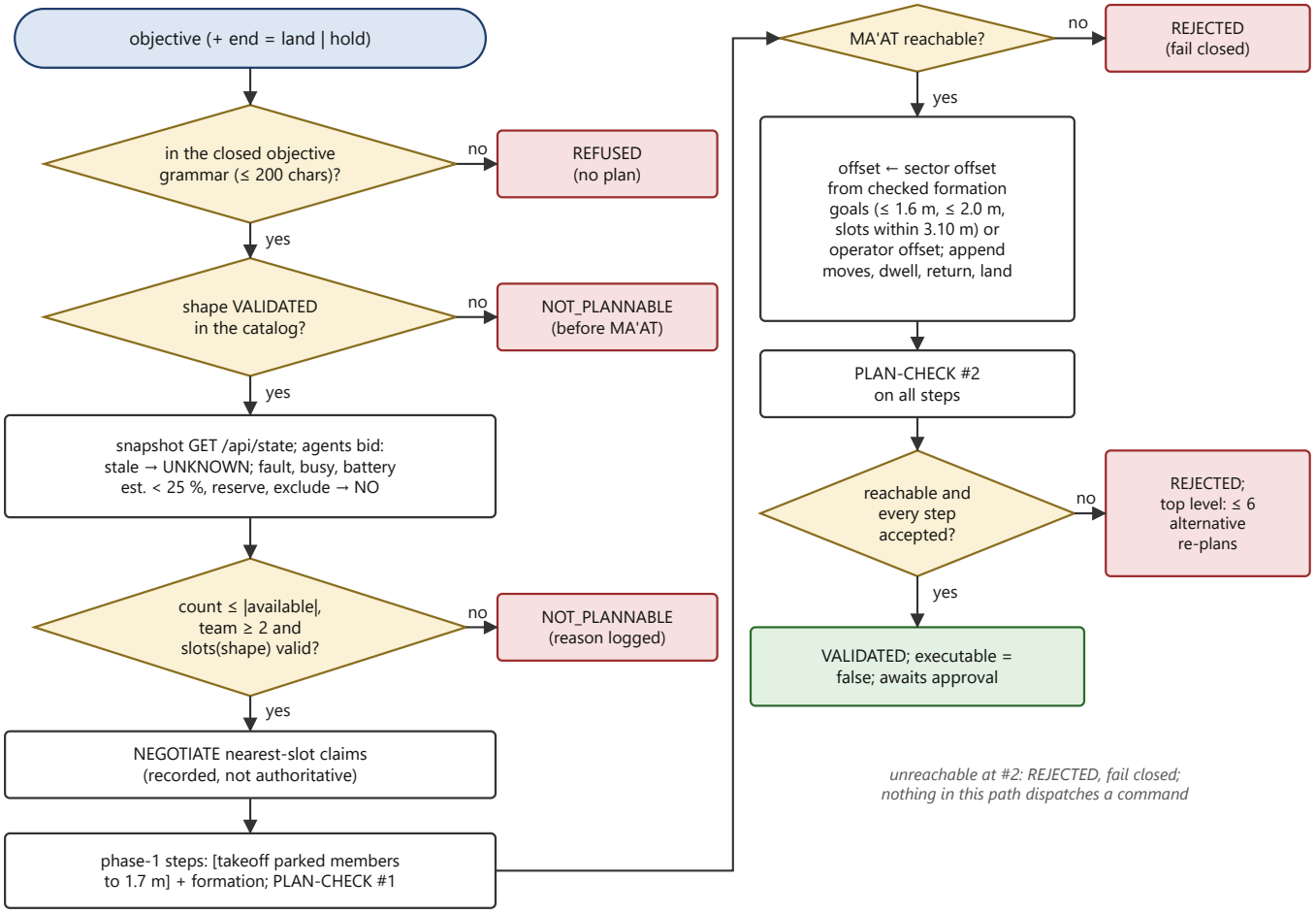


Figure 1. Planning flowchart of `Coordinator.plan`. Nothing in this path dispatches a command; both MA'AT calls are the read-only `POST /api/plan/check` (Algorithm 10).

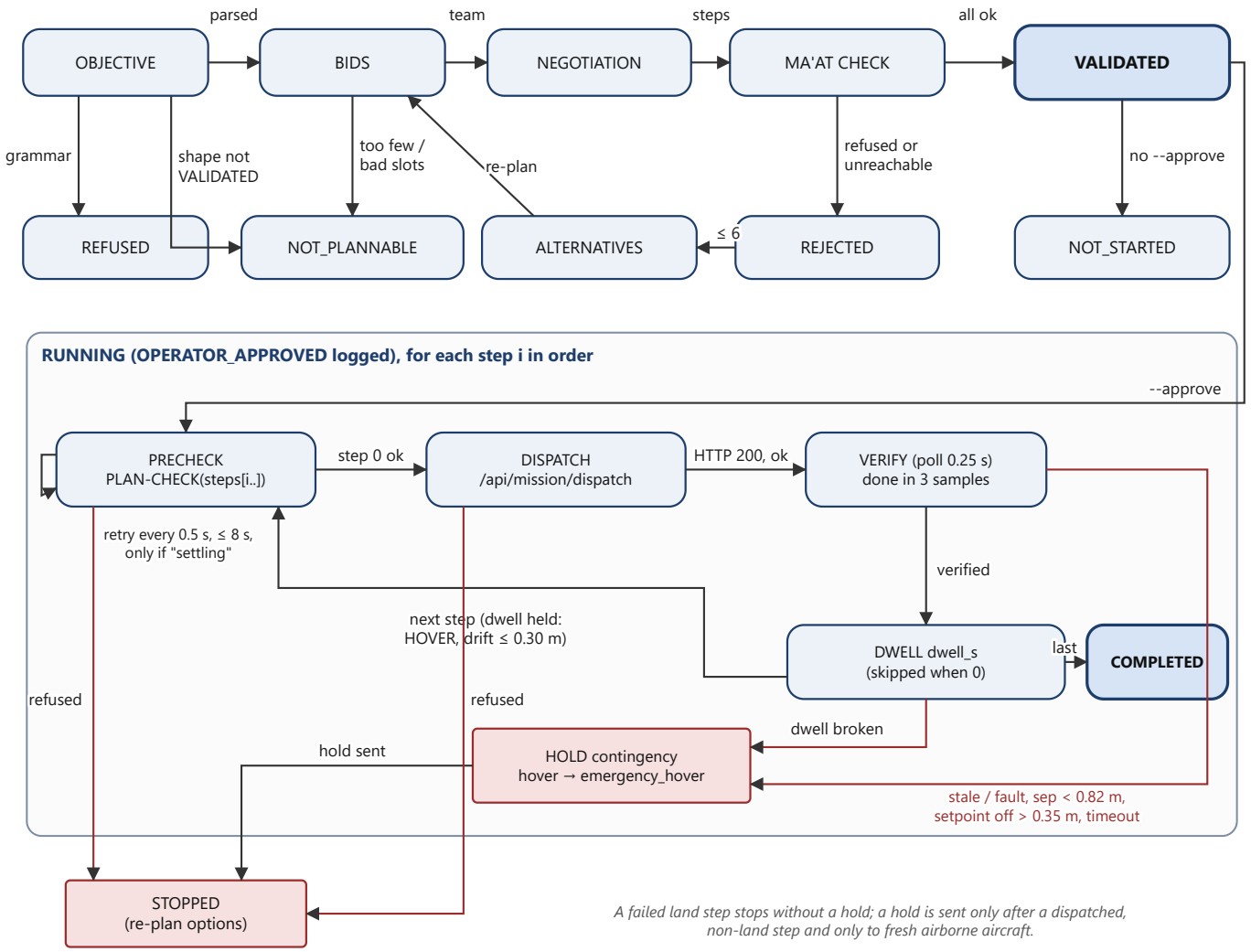


Figure 2. Mission lifecycle: objective → bids → negotiation → MA'AT plan check → operator approval → per-step execution with telemetry verification. Plan statuses are NOT_PLANNABLE, VALIDATED, REJECTED (REFUSED means `parse_objective` raised and no plan exists); report statuses are NOT_STARTED, RUNNING, COMPLETED, STOPPED. Alternatives re-enter the lifecycle at depth 1 and need their own approval.

5. Parameters

Parameter	Value
Arena / planning limit	3.35 m (supervisor) / 3.10 m (coordinator margin 0.25 m)
Sector distance, max move	1.6 m; 2.0 m; offsets quantised down to 0.05 m
Take-off altitude	1.7 m
Low simulated battery	< 25 %; drain 2 %/min airborne (estimate only)
Dwell	inspect 8 s; relay / hold 6 s; survey 6 s at each edge
Verify timeouts	takeoff 45 s, formation 45 s, move 40 s, hover 10 s, land 60 s; 3 consecutive samples; poll 0.25 s
Deviation / drift / separation	0.35 m setpoint deviation; 0.30 m dwell drift; 0.82 m observed separation
Pre-check retry	≤ 8 s, every 0.5 s, only for "not settled / maneuvering / wait for stable"
Alternatives	≤ 6 re-plans, depth 1

6. Complexity

Planning: two PLAN-CHECK calls (each dominated by formation assignment, Algorithm 5) plus, on rejection, up to 6 further full plans: ≤ 14 dry runs. Negotiation: ≤ |team|+1 rounds of O(|team|·|C|). Execution: one dry run per step attempt plus polling; wall time is dominated by flight time and timeouts.

7. Properties

Guaranteed

- Nothing is dispatched without both a VALIDATED status and explicit approval (`approve is True`); chat and planning paths never call EXECUTE.
- Each step is re-validated against the live state immediately before dispatch, through the supervisor's own code; MA'AT unavailability fails closed (REJECTED).
- Completion is judged from telemetry, not acknowledgements; a step must hold its completion condition for 3 consecutive samples.
- On any deviation after dispatch the fleet is commanded to hold, escalating to the symbolic emergency hover if the hold is refused.
- Only catalogue shapes with status VALIDATED can be planned. Planning is deterministic for a given snapshot (test `test_plan_is_deterministic`).
- The supervisor's assignment overrides the agents' negotiated slots; the transcript states this explicitly.

Not guaranteed

- "Agents", "bids" and "negotiation" are deterministic rules; the negotiation result does not influence the flown assignment.
- Battery values are simulated estimates, not measurements.
- Verification samples at 0.25 s; transient violations between samples are not seen. Aborting does not undo partial effects.
- Alternative search is bounded and heuristic; failure to find one does not mean none exists.
- A plan validated now may become invalid later; only the per-step re-check protects execution.

8. Testing and measured results

- `acceptance/tests/test_mission_coordinator.py` (26 tests): agents (`test_faulted_stale_lowbattery_reserve_busy`, `test_negotiation_yield_is_deterministic`), coordinator (`test_objective_grammar`, `test_inspect_north_plan_structure_and_graphs`, `test_unsafe_offset_rejected_with_safe_alternative`, `test_unvalidated_shape_refused_before_maat`, `test_maat_unavailable_fails_closed`), execution with `OfflineSupervisor(simulate_dispatch=True)` (`test_requires_approval`, `test_simulated_execution_verified_and_logged`, `test_stops_when_state_changes_after_planning`, `test_deviation_holds_fleet`). Included in the offline suite (44 tests OK, run `run_20261111T052914Z`).
- Live end-to-end executions (`logs/missions/`, 7 files, all route LIVE, 2026-11-11): three operator-approved executions COMPLETED with every step verified from telemetry: `M20261111T064410Z-618bdf` "inspect north sector", 6/6 steps, observed minimum separation 1.36 m; `M20261111T064459Z-197f23`, 5/5 steps, 1.366 m; and `M20261111T064530Z-80f1c1-alt6`, a MA'AT alternative executed with its own approval after the original plan was REJECTED ("S2 move: Group move rejected: Osiris would pass within 1.0 m of Anubis"), 4/4 steps, minimum 0.981 m. One further plan was REJECTED for an operator offset of 2.0 m ("Group move would take Thoth outside the validated arena/altitude bounds"). No STOPPED live mission is recorded, so the abort/hold path is tested offline only. Alternatives are planned with logging disabled, so `-alt6` has no PLAN record.

- Catalogue state on 2026-11-12: 105 shapes VALIDATED, `pentomino_w` EXPERIMENTAL after its proximity holds (Algorithm 7), so an objective naming it is refused before MA'AT.

9. Comparison with alternatives and related work

Multi-robot task allocation is classically done by market mechanisms: the contract net protocol (announce, bid, award) [1], auction algorithms for the assignment problem [2], and decentralized consensus-based auctions such as CBBA [3]. In the taxonomy of [4], formation-slot assignment is a single-task-robot, single-robot-task, instantaneous assignment (ST-SR-IA) problem, which reduces to the linear assignment problem. The HU-MA'AT coordinator borrows the vocabulary (bids, negotiation) but is a centralized deterministic rule set whose authoritative assignment comes from the supervisor.

Approach	Mechanism	What it guarantees (under its assumptions)	Compared with ours
Contract net protocol [1]	A manager announces a task, contractors evaluate and bid, the manager awards a contract; contractors may sub-contract.	A general negotiation protocol for distributed problem solving; no optimality claim.	Ours: bids are availability flags with reasons and a distance; the coordinator selects the team (closest to the sector point); there are no contracts or sub-contracting.
Auction algorithm [2]	Bidders raise prices on their best objects by the bid increment; a relaxation method for the dual of the assignment problem.	Terminates with an assignment within n·ε of optimal; parallelisable.	Ours: the agents' nearest-slot negotiation is greedy (not optimal) and only recorded; the flown assignment is the supervisor's exhaustive permutation search with separation constraints (exact for ≤ 5 aircraft, factorial cost).
CBBA [3]	Each agent builds a bundle of tasks by greedy bidding, then resolves conflicts through local consensus with neighbours.	Converges to a conflict-free assignment with a performance guarantee for diminishing marginal gains; tolerant of inconsistent situational awareness.	Ours is centralized in one process with one telemetry snapshot; no communication model, no robustness to message loss, no decentralization.
HU-MA'AT coordinator (this document)	Closed objective grammar → deterministic bids → recorded negotiation → MA'AT dry run → operator approval → verified execution.	Properties of Section 7 (no dispatch without VALIDATED + approval; per-step live re-check; telemetry-verified completion).	—

Where ours is weaker. Allocation is centralized (single point of failure), uses distance as the only cost, does not re-allocate during execution (it stops and the operator re-plans), and its "negotiation" has no influence on the outcome. The alternative search is a bounded heuristic. **What it adds.** Every decision carries telemetry evidence and a source label, safety authority stays with the supervisor's own code path, and the human approval gate is enforced in code. These are properties the market-based methods above do not address, and they are the reason the market vocabulary is used here only descriptively.

10. Relation to other algorithms

Uses Algorithm 5 (slot geometry), Algorithm 10 (validation), and through the gateway Algorithms 6–9; escalates through Algorithm 2. SwarmChat (Algorithm 12) can show a plan (PLAN ONLY) but never executes it.

11. References

1. R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Transactions on Computers*, vol. C-29, no. 12, pp. 1104–1113, 1980, doi:10.1109/TC.1980.1675516. `https://doi.org/10.1109/TC.1980.1675516`
2. D. P. Bertsekas, "The auction algorithm: A distributed relaxation method for the assignment problem," *Annals of Operations Research*, vol. 14, pp. 105–123, 1988, doi:10.1007/BF02186476. `https://www.mit.edu/~dimitrib/TheAuctionAP.pdf`
3. H.-L. Choi, L. Brunet and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009, doi:10.1109/TRO.2009.2022423. `http://hdl.handle.net/1721.1/52330`

4. B. P. Gerkey and M. J. Matarić, “A formal analysis and taxonomy of task allocation in multi-robot systems,” *International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004, doi:10.1177/0278364904045564.
`https://doi.org/10.1177/0278364904045564`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.